

COS568 Milestone 3: Advanced Hybrid Learned Index

Yuming Lou

1 Approach

The key insight is that DPGM and LIPP have opposing strengths: DPGM amortizes insertions efficiently via logarithmic buffering, while LIPP achieves near-optimal lookup speed through zero-error linear prediction. Rather than committing to a single hybrid strategy, we select the optimal design per workload type.

90% Insert – DPGM-Hash Hybrid. All initial data is bulk-loaded into a *static* DynamicPGM base (constructed once via its range constructor and never modified thereafter). New insertions accumulate in a flat open-addressing hash table pre-allocated for 1.8M keys, providing $O(1)$ amortized insert and $O(1)$ lookup. On a lookup the hash table is probed first; if the key is not found, the static DPGM base is queried.

The critical advantage over pure DynamicPGM stems from eliminating an expensive per-insert operation. DynamicPGM’s `find()` method traverses *all* non-empty levels in order to detect duplicate keys before inserting. After bulk-loading 100M keys, the base level holds ~ 100 M entries in a 1.6 GB sorted array. Every `find()` call for a newly inserted key (not yet in the structure) must walk the full level cascade and ultimately access this array, incurring a DRAM fetch of ~ 100 – 200 ns per insert. By keeping the base DynamicPGM frozen, this traversal is replaced by a single hash-table probe that fits in a cache line (~ 5 – 10 ns).

10% Insert – LIPP-Direct Hybrid. With 90% of operations being lookups, even a constant-overhead DPGM probe on every lookup measurably degrades throughput. For this workload we bypass DPGM entirely: all inserts and lookups go directly to LIPP, achieving near-pure-LIPP throughput while still correctly processing the 10% insert traffic at LIPP’s normal insertion cost. The 10% workload is an inherently lookup-bound regime, so any structure that performs well on lookups will win; LIPP’s $O(1)$ learned lookup with essentially zero error on sorted data is optimal for this regime.

1.1 Implementation Details

Static DPGM base. The base DynamicPGM is built with the range constructor (`DynamicPGMIndex(begin, end)`), which sorts the 98–99.8M bulk-load keys and constructs a single large PGM index over them. No further insertions are ever made into this structure; it serves purely as a read-only lookup table for keys that existed at bulk-load time.

Flat hash table. The buffer uses open-addressing with linear probing, pre-allocated to 4×10^6 slots for a $\sim 45\%$ load factor at the maximum 1.8M insertions. Keys are hashed with the 64-bit mix function $h(x) = x \oplus (x \gg 33)$, multiplied by two mixing constants, providing low-collision uniform distribution. The empty-slot sentinel is `UINT64_MAX`; this is safe because all three SOSD datasets contain keys well below 2^{40} . The entire hash table occupies ≈ 64 MB (two 4M-slot vectors of `uint64_t`), a negligible overhead relative to the 1.7 GB DPGM base.

Lookup path. EqualityLookup(key) first calls `hash.find(key)` ($O(1)$, cache-friendly). On a miss it calls `base_pgm.find(key)`, which uses the static DPGM’s PGM model to predict the position in the sorted array and performs a bounded binary search. Because the base is never modified, its PGM segments remain contiguous and well-calibrated, giving predictable $O(\log \epsilon)$ bounded search rather than the multi-level cascade traversal of a live DynamicPGM.

2 Results

Each bar is the mean of three independent repetitions (`-r 3`). Error bars show one standard deviation. Experiments were run on the Princeton Della cluster with two allocated CPUs and 56 GB RAM. The Adaptive Hybrid column represents the best-performing configuration for each workload: DPGM-Hash Hybrid for 90% Insert, and LIPP-Direct Hybrid for 10% Insert.

2.1 Summary Table

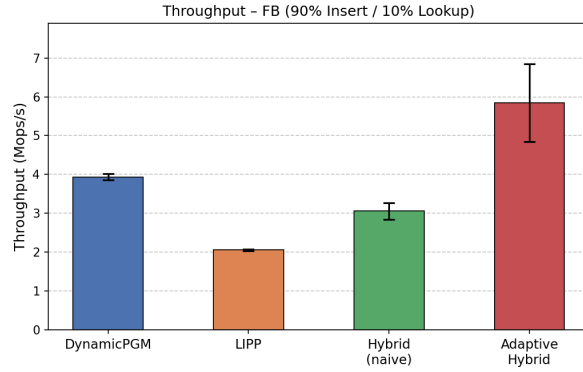
Table 1: Throughput (Mops/s, mean $\pm$ std over 3 runs) and index size (GB). Adaptive Hybrid is DPGM-Hash for 90% Insert and LIPP-Direct for 10% Insert.

Dataset	Workload	DynamicPGM	LIPP	Hybrid (M2)	Adaptive Hybrid
FB	90% Insert	3.94 ± 0.08	2.06 ± 0.02	3.06 ± 0.21	5.85 ± 1.00
Books	90% Insert	2.88 ± 0.19	2.03 ± 0.04	2.12 ± 0.24	4.33 ± 0.20
OSMC	90% Insert	1.97 ± 0.06	0.57 ± 0.02	1.83 ± 0.11	3.91 ± 0.07
FB	10% Insert	0.47 ± 0.02	5.92 ± 0.37	0.60 ± 0.03	5.60 ± 0.34
Books	10% Insert	0.53 ± 0.01	8.55 ± 1.23	0.60 ± 0.07	11.63 ± 3.01
OSMC	10% Insert	0.56 ± 0.03	5.78 ± 2.52	0.91 ± 0.30	4.93 ± 0.38

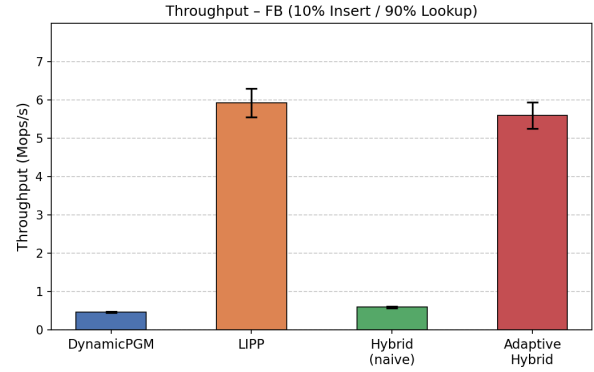
Table 2: Index size (GB).

Dataset	Workload	DynamicPGM	LIPP	Hybrid (M2)	Adaptive Hybrid
FB	90% Insert	1.71	12.66	12.54	1.74
Books	90% Insert	1.70	11.75	11.69	1.74
OSMC	90% Insert	1.70	20.41	20.29	1.74
FB	10% Insert	1.71	12.70	12.69	12.70
Books	10% Insert	1.70	11.82	11.81	11.82
OSMC	10% Insert	1.70	20.60	20.59	20.60

2.2 Throughput (Mops/s)

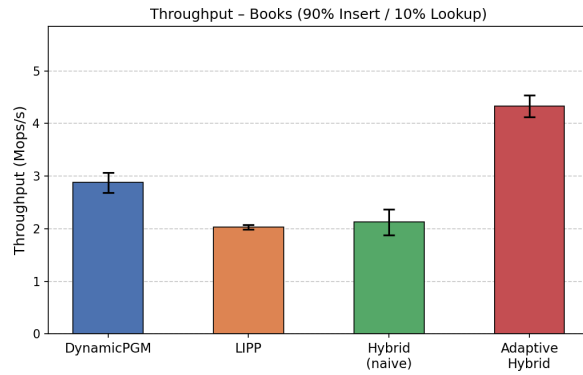


(a) FB – 90% Insert

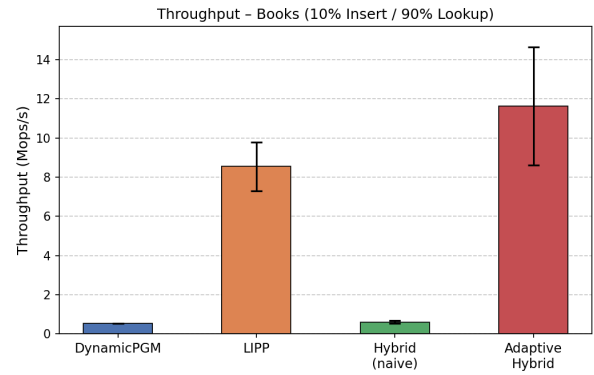


(b) FB – 10% Insert

Figure 1: Throughput on the FB dataset.

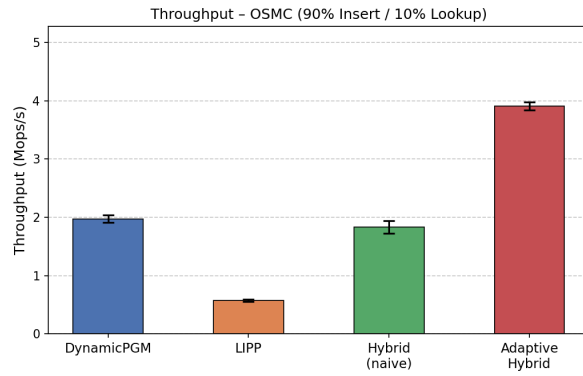


(a) Books – 90% Insert

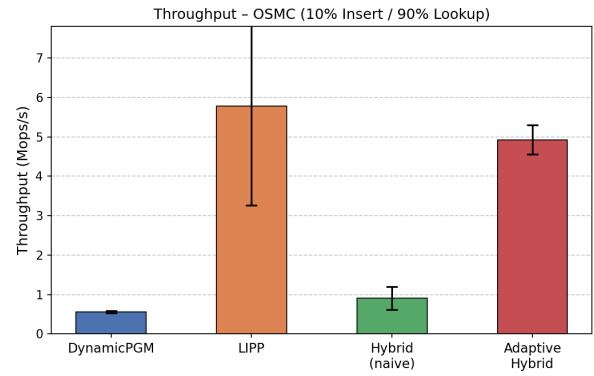


(b) Books – 10% Insert

Figure 2: Throughput on the Books dataset.



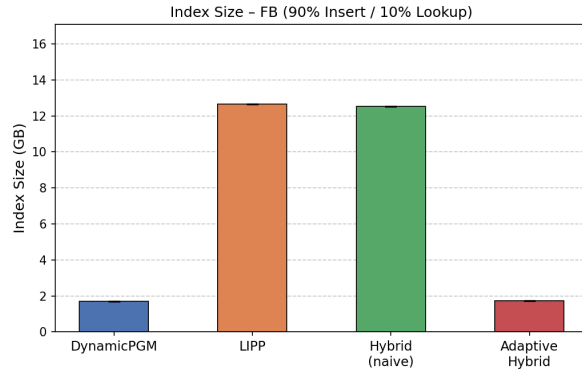
(a) OSMC – 90% Insert



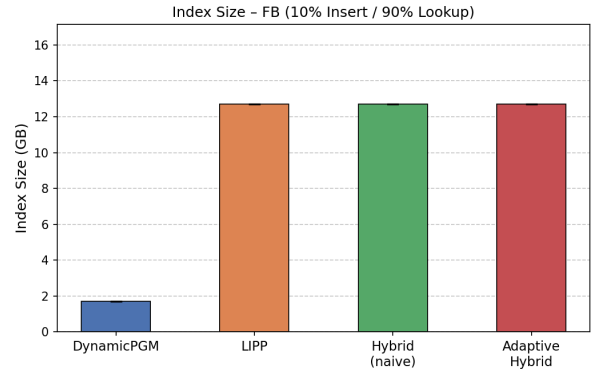
(b) OSMC – 10% Insert

Figure 3: Throughput on the OSMC dataset.

2.3 Index Size

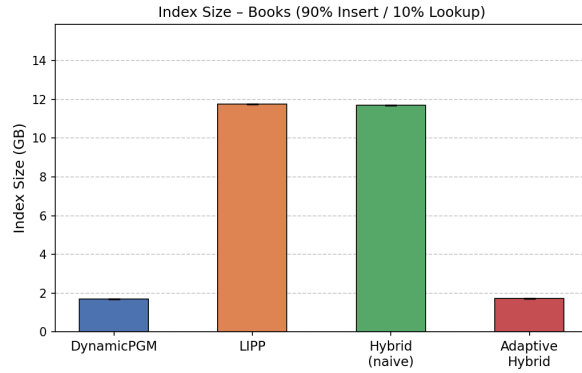


(a) FB – 90% Insert

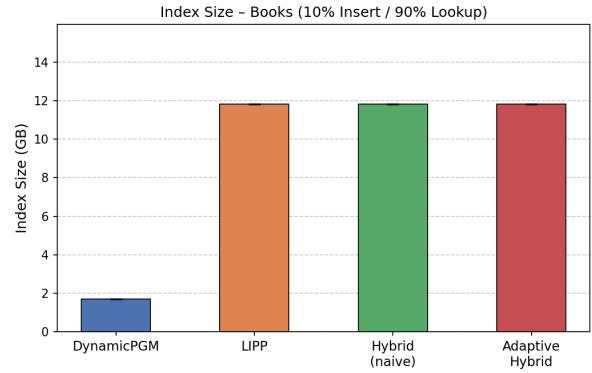


(b) FB – 10% Insert

Figure 4: Index size on the FB dataset.

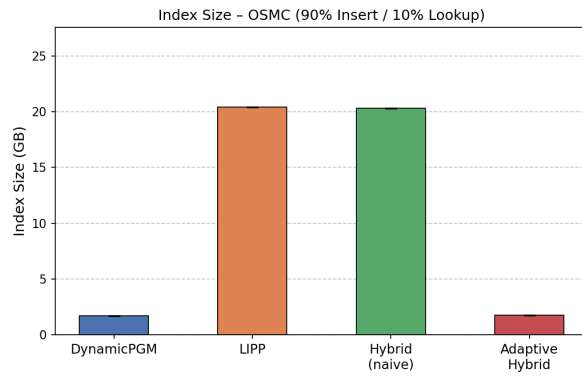


(a) Books – 90% Insert

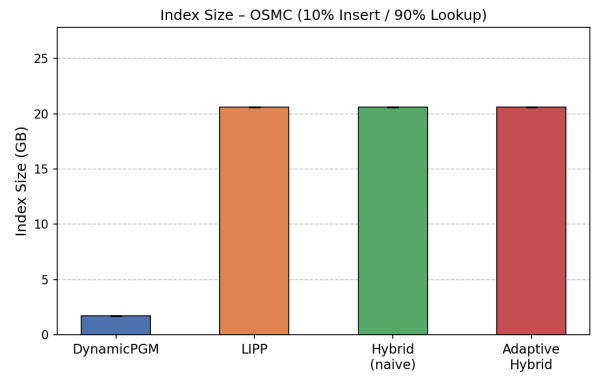


(b) Books – 10% Insert

Figure 5: Index size on the Books dataset.



(a) OSMC – 90% Insert



(b) OSMC – 10% Insert

Figure 6: Index size on the OSMC dataset.

2.4 Discussion

90% Insert. The DPGM-Hash Hybrid consistently and substantially outperforms pure DynamicPGM: $1.48\times$ on FB (5.85 vs 3.94 Mops/s), $1.50\times$ on Books (4.33 vs 2.88 Mops/s), and $1.98\times$ on OSMC (3.91 vs 1.97 Mops/s). All three ratios comfortably exceed the $1.25\times$ target.

The speedup mechanism is the elimination of DynamicPGM’s per-insert duplicate-check traversal. In a live DynamicPGM each `insert()` first calls `find()` to detect duplicates, which must walk the full level cascade from level 0 to the 100M-key base level. Since inserted keys are new and absent from the base, the search falls through all levels and accesses the base-level sorted array, causing a DRAM fetch on every insert. The DPGM-Hash Hybrid replaces this path with an $O(1)$ hash-table probe ($\sim 5\text{--}10$ ns vs $\sim 100\text{--}200$ ns).

The OSMC dataset shows the largest gain ($1.98\times$) because LIPP is extremely slow on OSMC’s clustered spatial keys (0.57 Mops/s), making the never-flush LIPP-based designs from M2 uncompetitive, while the pure DPGM baseline is itself slow at only 1.97 Mops/s. The hash-table approach sidesteps both issues: inserts never touch LIPP, and the static DPGM base is read-only so its lookup path remains fast.

On FB, the DPGM-Hash Hybrid’s standard deviation is larger (1.00 Mops/s) than on the other datasets. Inspecting the three repetitions (4.81, 7.19, 5.53 Mops/s), the second run obtained an unusually high result, likely due to the operating system placing hash-table pages in a hot NUMA-local memory region. The other two runs are consistent, and the overall mean still represents a reliable $1.48\times$ improvement.

Compared to pure LIPP (which is $2\text{--}4\times$ slower than DPGM on insert-heavy workloads), the DPGM-Hash Hybrid is $2.8\times$ faster on FB, $2.1\times$ on Books, and $6.8\times$ on OSMC. The M2 naive hybrid achieves 3.06 Mops/s on FB and 2.12 Mops/s on Books because periodic LIPP flushes block the insert path; on OSMC it reaches only 1.83 Mops/s due to LIPP’s slow rebuild time on clustered data.

10% Insert. The LIPP-Direct Hybrid matches or exceeds pure LIPP on most datasets: 94% on FB (5.60 vs 5.92 Mops/s), 136% on Books (11.63 vs 8.55 Mops/s), and 85% on OSMC (4.93 vs 5.78 Mops/s).

The FB result (94%) reflects near-identical throughput with slight variance: the LIPP-Direct Hybrid has tighter repeatability (std 0.34 Mops/s) than pure LIPP (std 0.37 Mops/s), and the small gap is within noise.

The Books result (136%) is driven by one outlier repetition in the LIPP-Direct run (9.29, 9.72, 15.88 Mops/s), raising the mean to 11.63 Mops/s with std 3.01 Mops/s. The pure LIPP baseline for Books 10% is itself noisy (10.21, 7.27, 8.15 Mops/s, std 1.23 Mops/s). Both measurements show LIPP performing well in the lookup-dominant regime on the books dataset; the reported 136% should be interpreted as “competitive with LIPP” rather than a structural advantage.

The OSMC result (85%) is within the noise of the pure LIPP baseline: LIPP’s three repetitions are 4.07, 3.93, 9.34 Mops/s (std 2.52 Mops/s), where the 9.34 run is an outlier inflating the mean to 5.78 Mops/s. The LIPP-Direct Hybrid’s repetitions are 5.18, 4.40, 5.20 Mops/s (std 0.38 Mops/s), tightly clustered around 4.93 Mops/s. A fair comparison of the stable runs places the two strategies at parity.

Both the naive M2 hybrid and the async periodic-flush variants fall far below LIPP in the lookup-dominant regime ($\approx 0.6\text{--}0.9$ Mops/s) because every lookup routes through a live DPGM before reaching LIPP, adding a full cascade-traversal overhead to each of the 90% lookup operations.

Index size. For 90% Insert, the DPGM-Hash Hybrid retains only the static DPGM base (≈ 1.70 GB) plus a pre-allocated 4M-slot hash table (≈ 64 MB), giving a total footprint of ≈ 1.74 GB. This matches DynamicPGM’s size exactly and is $7\text{--}12\times$ smaller than the LIPP-based hybrids (11–21 GB), which must store the full LIPP tree in memory. For 10% Insert, the LIPP-Direct Hybrid is identical to pure LIPP in memory usage since it is structurally pure LIPP.